EDS PRACTICAL

NAME – OMKAR NITIN SAKORE

PRN-202501110170

DIV -AL2 BATCH -A23

Practical 1

- **Practice Lab Assignment-1**

1.1.1. <u>Calculate Momentum</u>

```
m=float(input())
v=float(input())
p=m*v
print(f"{p:.2f}kgm/s")
```

1.1.2. Conditional Calculation Based on the Number of Digits

```
#write your code here...
n=int(input())
digits = len(str(abs(n)))
if digits == 1:
   print(n**2)
elif digits == 2:
   print(f"{n**(1/2):.2f}")
elif digits == 3:
   print (f"{n**(1/3):.2f}")
else:
  print("Invalid")
```

1.1.3. Days between Two Dates

```
from datetime import datetime
date1=input().strip()
date2=input().strip()
d1 = datetime.strptime(date1,"%Y-%m-%d")
d2 = datetime.strptime(date2,"%Y-%m-%d")
difference= d2 - d1
print(difference.days)
```

1.1.4. Reverse a Number

```
n = int(input())
rev = 0
while n>0:
    rev=rev*10+n%10
    n//=10
print(rev)
```

1.1.5. Multiplication Table

```
n = int(input())
for i in range(1, 11):
    print(f"{n} x {i} = {n*i}")
```

Lab Assignment

1.2.1. Pass or Fail

```
# write your code here
# Input number of courses
n = int(input())

# Input marks
marks = list(map(int, input().split()))
```

```
fail = False

# Check if any mark is less than 40
for m in marks:
    if m < 40:
        fail = True
        break

if fail:
    print("Fail")
else:
    aggregate = sum(marks) / n

    if aggregate > 75:
        grade = "Distinction"
    elif aggregate >= 60:
        grade = "First Division"
    elif aggregate >= 50:
```

```
        grade = "Second Division"
    else:
        grade = "Third Division"

    print(f"Aggregate Percentage: {aggregate:.2f}")
    print(f"Grade: {grade}")
```

1.2.2. Fibonacci Series

```
def fibonacci(n):
        if n==1:
                return 0
        elif n==2:
                return 1
        else:
                return fibonacci(n-1)+fibonacci(n-2)



n = int(input())
for i in range(1, n + 1):
```

```
    print(fibonacci(i), end=" ")
```

1.2.3. Pattern – 1

```
n=int(input())
for i in range(1,n+1):
    print( "* " *i)
```

1.2.4. Pattern – 2

```
n=int(input())
for i in range(1,n+1):
    for j in range(1,i+1):

        print(j,end=" ")
    print()
```

Practical 2

- **Practice Lab Assignment-2**

2.1.1. List operations

```
numbers=[]
while True:
```

```
print("1. Add")
print("2. Remove")
print("3. Display")
print("4. Quit")
choice = input("Enter choice: ")
if choice =="1":
    try:
        num=int(input("Integer: "))
        numbers.append(num)
        print("List after adding:",numbers)
    except ValueError:
        print("Invalid input")

elif choice=="2":
    if not numbers:
        print("List is empty")
    else:
        try:
            num=int(input("Integer: "))
```

```
                if num in numbers:
                    numbers.remove(num)
                    print("List after removing:",numbers)
                else:
                    print("Element not found")
            except ValueError:
                print("Invalid input")
    elif choice=="3":
        if not numbers:
            print("List is empty")
        else:
            print(numbers)
    elif choice=="4":
        break
    else:
        print("Invalid choice")
```

2.1.2. Dictionary Operations

```
# Initial dictionary with 10 predefined records
```

```
student = {
    1: "Amit",
    2: "Riya",
    3: "Kiran",
    4: "Neha",
    5: "Arjun",
    6: "Pooja",
    7: "Rahul",
    8: "Sneha",
    9: "Vikram",
    10: "Anjali"
}

# write your code here...
print("Original Dictionary:", student)
key = int(input())
value = input()
student[key] = value
print("After Insertion:", student)
```

```
key = int(input())
value = input()
if key in student:
    student[key] = value
print("After Update:", student)

key = int(input())
if key in student:
    student.pop(key)
print("After Deletion:", student)

print("Traversing Dictionary:")
for k, v in student.items():
    print(k, ":", v)
```

Lab Assignment

2.2.1. Linear search Technique

```
arr = list(map(int, input().split()))
```

```
key = int(input())

found = False
for i in range(len(arr)):
    if arr[i] == key:
        print(i)
        found = True
        break

if not found:
    print("Not found")
```

2.2.2. Captain of the Team

```
heights=list(map(int,input().split()))
tallest=max(heights)
print(tallest)
```

Practical 3

- **Practice Lab Assignment-3**

```
import numpy as np


# write your code here...
rows, cols = map(int, input().split())
if rows == 0 or cols == 0:
    arr = np.array([]).reshape(rows, cols)
else:
    elements = []
    for _ in range(rows):
        row = list(map(int, input().split()))
        elements.extend(row)

    arr = np.array(elements).reshape(rows, cols)


print(arr)
print(arr.ndim)
print(arr.shape)
print(arr.size)
```

Lab Assignment

3.2.1. Numpy: Matrix Operations

```
import numpy as np


# Input matrices
print("Enter Matrix A:")
matrix_a = np.array([list(map(int, input().split())) for i
in range(3)])


print("Enter Matrix B:")
matrix_b = np.array([list(map(int, input().split())) for i
in range(3)])



# Addition
print("Addition (A + B):")
print(matrix_a+matrix_b)
# Subtraction
print("Subtraction (A - B):")
```

```
print(matrix_a-matrix_b)
# Multiplication (element-wise)
print("Element-wise Multiplication (A * B):")
print(matrix_a*matrix_b)
# Matrix multiplication (dot product)
print("A dot B:")
print(np.dot(matrix_a,matrix_b))
# Transpose
print("Transpose of A:")
print(matrix_a.T)
```

3.2.2. Numpy: Horizontal and Vertical Stacking of Arrays

```
import numpy as np

# Input matrices
print("Enter Array1:")
arr1 = np.array([list(map(int, input().split())) for i in range(3)])
```

```
print("Enter Array2:")
arr2 = np.array([list(map(int, input().split())) for i in range(3)])

# Perform horizontal stacking (hstack)
print("Horizontal Stack:")
print(np.hstack((arr1,arr2)))



# Perform vertical stacking (vstack)
print("Vertical Stack:")
print(np.vstack((arr1,arr2)))
```

3.2.3. Numpy: Custom Sequence Generation

```
import numpy as np

# Take user input for the start, stop, and step of the sequence
start = int(input())
stop = int(input())
step = int(input())
```

```
# Generate the sequence using np.arange()
sequence = np.arange(start,stop,step)
# Print the generated sequence
print(sequence)
```

3.2.4. Numpy: Arithmetic and Statistical Operations, Mathematical Operations, Bitwise Operators

```
import numpy as np


def array_operations(A, B):


    # Convert A and B to NumPy arrays
    A = np.array(A)
    B = np.array(B)
    # Arithmetic Operations
    sum_result = A+B
    diff_result = A-B
    prod_result = A*B
```

```
    # Statistical Operations
    mean_A = np.mean(A)
    median_A = np.median(A)
    std_dev_A = np.std(A)

    # Bitwise Operations
    and_result = A&B
    or_result = A|B
    xor_result = A^B

  # Output results with one space between each element
    print("Element-wise Sum:", ' '.join(map(str, sum_result)))
    print("Element-wise Difference:", ' '.join(map(str, diff_result)))
    print("Element-wise Product:", ' '.join(map(str, prod_result)))

    print(f"Mean of A: {mean_A}")
```

```
    print(f"Median of A: {median_A}")
    print(f"Standard Deviation of A: {std_dev_A}")

    print("Bitwise AND:", ' '.join(map(str, and_result)))
    print("Bitwise OR:", ' '.join(map(str, or_result)))
    print("Bitwise XOR:", ' '.join(map(str, xor_result)))

A = list(map(int, input().split()))  # Elements of array A
B = list(map(int, input().split()))  # Elements of array B
array_operations(A, B)
```

3.2.5. Numpy: Copying and Viewing Arrays

```
import numpy as np

inputlist = list(map(int,input().split(" ")))

# Original array
original_array = np.array(inputlist)
```

```
# Create a view
view_array = original_array.view()

# Create a copy
copy_array = original_array.copy()

# Modify the view
view_array[0] = 99
print("Original array after modifying view:", original_array)
print("View array:", view_array)

# Modify the copy
copy_array[1] = 88
print("Original array after modifying copy:", original_array)
print("Copy array:", copy_array)
```

3.2.6. Numpy: Searching, Sorting, Counting, Broadcasting

```
import numpy as np

# Input array from the user
array1 = np.array(list(map(int, input().split())))

# Searching
search_value = int(input("Value to search: "))
count_value = int(input("Value to count: "))
broadcast_value = int(input("Value to add: "))

indices = np.where(array1 == search_value)[0]
print(indices)

# Count occurrences
count = np.count_nonzero(array1 == count_value)
print(count)
```

```
# Broadcasting addition
print(array1 + broadcast_value)

# Sorted array
print(np.sort(array1))
```

3.2.7. Student Data Analysis and Operations

```
import numpy as np

a = np.loadtxt("Sample.csv", delimiter=',',
skiprows=1)

# 1. Print all student details
print("All student Details:\n",a)

# 2. print total students

print("Total Students:",a.shape[0])

# 3. Print all student Roll numbers
```

```
print("All Student Roll Nos",a[:, 0])

# 4. Print subject 1 marks
print("Subject 1 Marks",a[:,1])

# 5. print minimum marks of Subject 2
print("Min marks in Subject 2",np.min(a[:, 2]))

# 6. print maximum marks of Subject 3
print("Max marks in Subject 3",np.max(a[:, 3]))

# 7. Print All subject marks
print("All subject marks:",a[:, 1:])

# 8. print Total marks of students
total_marks=np.sum(a[:, 1:],axis=1)
print("Total Marks",total_marks)

# 9. print average marks of each student
```

```
print(np.round(np.mean(a[:, 1:], axis=1), 1))

# 10. print average marks of each subject
print("Average Marks of each subject",
np.round(np.mean(a[:, 1:], axis=0), 1))

# 11. print average marks of S1 and S2
print("Average Marks of S1 and S2",
np.round(np.mean(a[:, 1:3], axis=0), 1))

# 12. print average marks of S1 and S3
print("Average Marks of S1 and S3",
np.round(np.mean(a[:, [1, 3]], axis=0), 1))

# 13. print Roll number who got maximum marks in
Subject 3
print("Roll no who got maximum marks in Subject 3",
a[np.argmax(a[:, 3]), 0])
```

14. print Roll number who got minimum marks in Subject 2

```
print("Roll no who got minimum marks in Subject 2", a[np.argmin(a[:, 2]), 0])
```

15. print Roll number who got 24 marks in Subject 2

```
print("Roll no who got 24 marks in Subject 2", a[a[:, 2] == 24][:, [0]])
```

16. print count of students who got marks in Subject 1 < 40

```
print("Count of students who got marks in Subject 1 < 40", np.sum(a[:, 1] < 40))
```

17. print count of students who got marks in Subject 2 > 90

```
print("Count of students who got marks in Subject 2 > 90:", np.sum(a[:, 2] > 90))
```

```
# 18. print count of students in each subject who got marks >= 90
print("Count of students in each subject who got marks >= 90:", np.sum(a[:, 1:] >= 90, axis=0))

# 19. print count of subjects in which each student got marks >= 90
print("Roll no:", a[:, 0])
print("Count of subjects in which student got marks >= 90:", np.sum(a[:, 1:] >= 90, axis=1))

# 20. Print S1 marks in ascending order
print(np.sort(a[:, 1]))

# 21. Print rows where S1 marks >= 50 and <= 90
filtered=(a[(a[:, 1] >= 50) & (a[:, 1] <= 90)])
print(filtered)

print(a)
# 22. Print the index position of marks 79
```

```
print((np.array(np.where(a[:, 1]== 79)[0],
dtype=np.int32),))
```

Practical 4

- ## Practice Lab Assignment-4

4.1.1. Pandas - series creation and manipulation

```
import pandas as pd


# Take inputs from the user to create a list of
numbers
numbers = list(map(int, input().split()))


# Grouping by even and odd numbers and
calculating the mean
grouped = pd.Series(numbers).groupby(1 -
(pd.Series(numbers) %2)).mean()
```

```
# Display the mean of even and odd numbers with labels
grouped.index = ['Even' if is_even else 'Odd' for is_even in grouped.index]
print("Mean of even and odd numbers:")
print(grouped)
```

4.1.2. Dictionary to dataframe

```
import pandas as pd

# Provided dictionary of lists
data = {
    'Name': ['Alice', 'Bob', 'Charlie'],
    'Age': [25, 30, 35],
}

# Convert the dictionary to a DataFrame
df = pd.DataFrame(data)

# Display the original DataFrame
```

```
print("Original DataFrame:")
print(df)

name = input("New name: ")
age = int(input("New age: "))
df.loc[len(df)] = [name, age]

df= df.copy()


# Display the DataFrame after adding a new row
print("After adding a row:\n",df)

idx = int(input("Index of row to modify: "))
new_age = int(input("New age: "))
df.loc[idx, 'Age'] = new_age
```

```
# Display the DataFrame after modifying a row
print("After modifying a row:")
print(df)
del_idx = int(input("Index of row to delete: "))
df = df.drop(del_idx)
df = df.reset_index(drop=True)



# Display the DataFrame after deleting a row
print("After deleting a row:")
print(df)

# Adding a new column
genders = input("Enter genders separated by space:
").split()

if len(genders) == len(df):
    df['Gender'] = genders
```

```
# Display the DataFrame after adding a new column
print("After adding a new column:")
print(df)

# Modifying a column
df['Name'] = df['Name'].str.upper()


# Display the DataFrame after modifying a column
print("After modifying a column:")
print(df)

# Deleting a column
df = df.drop('Age', axis=1)


# Display the DataFrame after deleting a column
```

```
print("After deleting a column:")
print(df)
```

4.1.3. Student Information

```
import pandas as pd

# Read the text file into a DataFrame
file = input()
data = pd.read_csv(file, sep="\s+", header=None,
names=["Name", "Age", "Grade"])


# write your code here..
print("First five rows:")
print(data.head())

avg_age = round(data["Age"].mean(), 2)
print(f"Average age: {avg_age}")
```

```
filtered = data[data["Grade"] <= 'B']

print("Students with a grade up to B")
print(filtered)
```

Lab Assignment

4.2.1. Month with the Highest Total Sales

```
import pandas as pd

# Prompt the user for the file name
file_name = input()

# Load the data
df = pd.read_csv(file_name)
```

```
# Find the month with the highest total sales
best_month = (df.assign(Date=pd.to_datetime(df['Date']),
month=lambda x: x['Date'].dt.to_period('M').astype(str),
Sales=lambda x: x['Quantity'] * x['Price'])
.groupby('month')['Sales'].sum().idxmax())


highest_sales = (df.assign(Date=pd.to_datetime(df['Date']),
month=lambda x: x['Date'].dt.to_period('M').astype(str),
Sales=lambda x: x['Quantity'] * x['Price'])
.groupby('month')['Sales'].sum().max())
```

```
print(f"Best month: {best_month}")
print(f"Total sales: ${highest_sales:.2f}")
```

4.2.2. Best Selling Product

```
import pandas as pd

# Prompt the user for the file name
file_name = input()

# Load the data
df = pd.read_csv(file_name)


# Find the product with the highest total
quantity sold
```

```
best_product =
df.groupby('Product')['Quantity'].sum().idx
max()
highest_quantity =
df.groupby('Product')['Quantity'].sum().max
()

# Display the result
print(f"Best selling product:
{best_product}")
print(f"Total quantity sold:
{highest_quantity}")
```

4.2.3. City that Sold the Most Products

```
import pandas as pd

# Prompt the user for the file name
```

```
file_name = input()

# Load the data
df = pd.read_csv(file_name)

# write the code..
best_city =
df.groupby('City')['Quantity'].sum().idxmax()
# Display the result
print(f"City sold the most products:
{best_city}")
```

4.2.4. Most Frequently Sold Product Pairs

```
import pandas as pd
from itertools import combinations
from collections import Counter
```

```
# Prompt user to input the file name
file_name = input()

# Read data from the specified CSV file
df = pd.read_csv(file_name)
grouped =
df.groupby('Date')['Product'].apply(list)
pairs = []
for items in grouped:
    pairs +=
list(combinations(sorted(set(items)), 2))

pair_counts = Counter(pairs)

max_count = max(pair_counts.values())
for pair, count in pair_counts.items():
    if count == max_count:
```

```
        print(f"{pair[0]} and {pair[1]}: {count} times")
```

4.2.5. Titanic Dataset Analysis and Data Cleaning

```
import pandas as pd
import numpy as np

# Load the Titanic dataset
data = pd.read_csv('Titanic-Dataset.csv')

print(data.head())

# 2. Display the last 5 rows of the dataset
print(data.tail())

# 3. Get the shape of the dataset
print(data.shape)
```

```
# 4. Get a summary of the dataset (info)
print(data.info())

# 5. Get basic statistics of the dataset
print(data.describe())

# 6. Check for missing values
print(data.isnull().sum())

# 7. Fill missing values in the ‘Age’ column with the median age
data['Age'].fillna(data['Age'].median(), inplace=True)

# 8. Fill missing values in the ‘Embarked’ column with the mode
```

```
data['Embarked'].fillna(data['Embarked'].mode()[0], inplace=True)

# 9. Drop the ‘Cabin’ column due to many missing values
data.drop('Cabin', axis=1, inplace=True)

# 10. Create a new column 'FamilySize’ by adding ‘SibSp' and ‘Parch'
data['FamilySize'] = data['SibSp'] + data['Parch']
```

4.2.6. Titanic Dataset Analysis and Data Cleaning – 2

```
import pandas as pd
import numpy as np

# Load the Titanic dataset
```

```
data = pd.read_csv('Titanic-Dataset.csv')
data['FamilySize'] = data['SibSp'] +
data['Parch']

# 1. Create a new column ‘IsAlone' (1 if
alone, 0 otherwise)
data['IsAlone'] =
np.where(data['FamilySize'] == 0, 1, 0)

# 2. Convert ‘Sex' to numeric (male: 0,
female: 1)
data['Sex'] = data['Sex'].map({'male': 0,
'female': 1})

# 3. One-hot encode the ‘Embarked'
column
data = pd.get_dummies(data,
columns=['Embarked'])
```

```
# 4. Get the mean age of passengers
print(data['Age'].mean())

# Additional outputs (as per test case)
print(data['Fare'].median())
print(data['Pclass'].value_counts())
print(data['Sex'].value_counts())
print(data['Survived'].value_counts())
print(data['Survived'].mean())
print(data.groupby('Sex')['Survived'].mean()
)
```

4.2.7. Titanic Dataset Analysis and Data Cleaning – 3

```
import pandas as pd
import numpy as np
```

```
# Load the Titanic dataset
data = pd.read_csv('Titanic-Dataset.csv')
data['FamilySize'] = data['SibSp'] + data['Parch']
data['IsAlone'] = np.where(data['FamilySize'] > 0, 0, 1)
data = pd.get_dummies(data, columns=['Embarked'], drop_first=True)

# 1. Calculate the survival rate by class
print(data.groupby('Pclass')['Survived'].mean())

# 2. Calculate the survival rate by embarked location
print(data.groupby('Embarked_S')['Survived'].mean())
```

```
# 3. Calculate the survival rate by family size
print(data.groupby('FamilySize')['Survived'].mean())

# 4. Calculate the survival rate by being alone
print(data.groupby('IsAlone')['Survived'].mean())

# 5. Get the average fare by class
print(data.groupby('Pclass')['Fare'].mean())

# 6. Get the average age by class
print(data.groupby('Pclass')['Age'].mean())

# 7. Get the average age by survival status
```

```
print(data.groupby('Survived')['Age'].mean()
)

# 8. Get the average fare by survival status
print(data.groupby('Survived')['Fare'].mean(
))

# 9. Get the number of survivors by class
print(data[data['Survived'] ==
1]['Pclass'].value_counts())

# 10. Get the number of non-survivors by
class
print(data[data['Survived'] ==
0]['Pclass'].value_counts())
```

4.2.8. Titanic Dataset Analysis and Data Cleaning – 4

```
import pandas as pd
import numpy as np

# Load the Titanic dataset
data = pd.read_csv('Titanic-Dataset.csv')
data = pd.get_dummies(data,
columns=['Embarked'], drop_first=True)

# 1. Get the number of survivors by gender
print(data[data['Survived'] ==
1]['Sex'].value_counts())

# 2. Get the number of non-survivors by
gender
print(data[data['Survived'] ==
0]['Sex'].value_counts())
```

```
# 3. Get the number of survivors by embarked location
print(data[data['Survived'] == 1]['Embarked_S'].value_counts())

# 4. Get the number of non-survivors by embarked location
print(data[data['Survived'] == 0]['Embarked_S'].value_counts())

# 5. Percentage of children (Age < 18) who survived
children = data[data['Age'] < 18]
print(children['Survived'].mean())

# 6. Percentage of adults (Age >= 18) who survived
adults = data[data['Age'] >= 18]
```

```
print(adults['Survived'].mean())

# 7. Median age of survivors
print(data[data['Survived'] ==
1]['Age'].median())

# 8. Median age of non-survivors
print(data[data['Survived'] ==
0]['Age'].median())

# 9. Median fare of survivors
print(data[data['Survived'] ==
1]['Fare'].median())

# 10. Median fare of non-survivors
print(data[data['Survived'] ==
0]['Fare'].median())
```

Practical 5

- **Practice Lab Assignment-5**

5.1.1. Stacked Plot

```
import matplotlib.pyplot as plt
import pandas as pd

# Data for Months and Temperature for three cities
data = {
    'Month': ['January', 'February', 'March', 'April', 'May', 'June', 'July', 'August', 'September', 'October', 'November', 'December'],
```

```
    'City_A_Temperature': [5, 7, 10, 13, 17, 20, 22, 21, 18, 12, 8, 6],
    'City_B_Temperature': [2, 3, 5, 6, 10, 14, 16, 17, 12, 9, 5, 3],
    'City_C_Temperature': [3, 4, 6, 8, 9, 12, 15, 14, 10, 7, 4, 2]
}



df = pd.DataFrame(data)


plt.stackplot(df['Month'],
              df['City_A_Temperature'],
              df['City_B_Temperature'],
              df['City_C_Temperature'])
```

```
plt.xlabel('Month')
plt.ylabel('Temperature')
plt.title('Temperature Variation')
plt.show()
```

Lab Assignment

5.2.1. Titanic Dataset

```
import pandas as pd
import matplotlib.pyplot as plt

# Load the Titanic dataset from the CSV file
df = pd.read_csv('titanic.csv')

# Set up the figure for 5 subplots
fig, axes = plt.subplots(3, 2, figsize=(12, 12))
```

```
# 1. Bar Plot - Passenger Class Distribution
pclass_counts =
df['Pclass'].value_counts().sort_index()
axes[0, 0].bar(pclass_counts.index,
pclass_counts.values, color='skyblue')
axes[0, 0].set_title('Passenger Class
Distribution')
axes[0, 0].set_xlabel('Pclass')
axes[0, 0].set_ylabel('Count')

# 2. Pie Chart - Gender Distribution
gender_counts =
df['Gender'].value_counts()
axes[0, 1].pie(gender_counts.values,
               labels=gender_counts.index,
               autopct='%1.1f%%',
```

```
                    colors=['lightblue',
'lightcoral'])
axes[0, 1].set_title('Gender Distribution')

# 3. Histogram - Age Distribution
axes[1, 0].hist(df['Age'], bins=8,
color='lightgreen', edgecolor='black')
axes[1, 0].set_title('Age Distribution')
axes[1, 0].set_xlabel('Age')
axes[1, 0].set_ylabel('Frequency')

# 4. Bar Plot - Survival Count
survival_counts =
df['Survived'].value_counts().sort_index()
axes[1, 1].bar(survival_counts.index,
survival_counts.values,
                    color=['lightblue', 'lightcoral'])
```

```
axes[1, 1].set_title('Survival Count')
axes[1, 1].set_xlabel('Survived (0 = No, 1 = Yes)')
axes[1, 1].set_ylabel('Count')

# 5. Scatter Plot - Fare vs Age
axes[2, 0].scatter(df['Age'], df['Fare'], color='orange')
axes[2, 0].set_title('Fare vs Age')
axes[2, 0].set_xlabel('Age')
axes[2, 0].set_ylabel('Fare')

# Hide the empty 6th subplot (bottom right)
axes[2, 1].set_visible(True)

plt.tight_layout()
plt.show()
```

5.2.2. Histogram of passenger information of Titanic

```
import pandas as pd
import matplotlib.pyplot as plt

# Load the Titanic dataset
data = pd.read_csv('Titanic-Dataset.csv')

# Data Cleaning
data['Age'].fillna(data['Age'].median(), inplace=True)
data['Embarked'].fillna(data['Embarked'].mode()[0], inplace=True)
data.drop('Cabin', axis=1, inplace=True)

# Convert categorical features to numeric
```

```
data['Sex'] = data['Sex'].map({'male': 0, 'female': 1})
data = pd.get_dummies(data, columns=['Embarked'], drop_first=True)

# Write your code here for Histogram
plt.hist(data['Age'], bins=30, edgecolor='k')
plt.xlabel('Age')
plt.ylabel('Frequency')
plt.title('Age Distribution')
plt.show()
```

5.2.3. Bar plot of survival rate of passengers

```
import pandas as pd
import matplotlib.pyplot as plt

# Load the Titanic dataset
```

```
data = pd.read_csv('Titanic-Dataset.csv')

# Data Cleaning
data['Age'].fillna(data['Age'].median(), inplace=True)
data['Embarked'].fillna(data['Embarked'].mode()[0], inplace=True)
data.drop('Cabin', axis=1, inplace=True)

# Convert categorical features to numeric
data['Sex'] = data['Sex'].map({'male': 0, 'female': 1})
data = pd.get_dummies(data, columns=['Embarked'], drop_first=True)

# Write your code here for Bar Plot for Survival Rate
```

```
data['Survived'].value_counts().sort_index()
.plot(kind='bar')
plt.title('Survival Count')
plt.xlabel('Survived')
plt.ylabel('Count')
plt.show()
```

5.2.4. Bar Plot for Survival by Gender

```
import pandas as pd
import matplotlib.pyplot as plt

# Load the Titanic dataset
data = pd.read_csv('Titanic-Dataset.csv')

# Data Cleaning
data['Age'].fillna(data['Age'].median(),
inplace=True)
```

```
data['Embarked'].fillna(data['Embarked'].mode()[0], inplace=True)
data.drop('Cabin', axis=1, inplace=True)

# Convert categorical features to numeric
data['Sex'] = data['Sex'].map({'male': 0, 'female': 1})
data = pd.get_dummies(data, columns=['Embarked'], drop_first=True)

# Write your code here for Bar Plot for Survival by Gender
survival_gender = data.groupby('Sex')['Survived'].value_counts().unstack()
survival_gender.plot(kind='bar', stacked=True)
plt.title('Survival by Gender')
```

```
plt.xlabel('Gender')
plt.ylabel('Count')
plt.legend(['Not Survived', 'Survived'])
plt.show()
```

5.2.5. Bar Plot for Survival by Pclass

```
import pandas as pd
import matplotlib.pyplot as plt

# Load the Titanic dataset
data = pd.read_csv('Titanic-Dataset.csv')

# Data Cleaning
data['Age'].fillna(data['Age'].median(),
inplace=True)
data['Embarked'].fillna(data['Embarked'].m
ode()[0], inplace=True)
data.drop('Cabin', axis=1, inplace=True)
```

```
# Convert categorical features to numeric
data['Sex'] = data['Sex'].map({'male': 0, 'female': 1})
data = pd.get_dummies(data, columns=['Embarked'], drop_first=True)

# Write your code here for Bar Plot for Survival by Pclass
survival_pclass = data.groupby('Pclass')['Survived'].value_counts().unstack()
survival_pclass.plot(kind='bar', stacked=True)
plt.title('Survival by Pclass')
plt.xlabel('Pclass')
plt.ylabel('Count')
plt.legend(['Not Survived', 'Survived'])
```

```
plt.show()
```

5.2.6. Bar Plot for Survival by Embarked

```
import pandas as pd
import matplotlib.pyplot as plt

# Load the Titanic dataset
data = pd.read_csv('Titanic-Dataset.csv')

# Data Cleaning
data['Age'].fillna(data['Age'].median(),
inplace=True)
data['Embarked'].fillna(data['Embarked'].m
ode()[0], inplace=True)
data.drop('Cabin', axis=1, inplace=True)

# Convert categorical features to numeric
```

```
data['Sex'] = data['Sex'].map({'male': 0, 'female': 1})
data = pd.get_dummies(data, columns=['Embarked'], drop_first=True)

# Write your code here for Bar Plot for Survival by Embarked
survival_embarked = data.groupby('Embarked_Q')['Survived'].value_counts().unstack()
survival_embarked.plot(kind='bar', stacked=True)
plt.title('Survival by Embarked')
plt.xlabel('Embarked')
plt.ylabel('Count')
plt.legend(['Not Survived', 'Survived'])
plt.show()
```

5.2.7. Box plot for Age Distribution

```
import pandas as pd
import matplotlib.pyplot as plt

# Load the Titanic dataset
data = pd.read_csv('Titanic-Dataset.csv')

# Data Cleaning
data['Age'].fillna(data['Age'].median(),
inplace=True)
data['Embarked'].fillna(data['Embarked'].m
ode()[0], inplace=True)
data.drop('Cabin', axis=1, inplace=True)

# Convert categorical features to numeric
data['Sex'] = data['Sex'].map({'male': 0,
'female': 1})
```

```
data = pd.get_dummies(data,
columns=['Embarked'], drop_first=True)

# Write your code here for Box Plot for Age
by Pclass
data.boxplot(column='Age', by='Pclass')
plt.title('Age by Pclass')
plt.suptitle('')
plt.xlabel('Pclass')
plt.ylabel('Age')
plt.show()
```

5.2.8. Box Plot for Age by Survived

```
import pandas as pd
import matplotlib.pyplot as plt

# Load the Titanic dataset
```

```
data = pd.read_csv('Titanic-Dataset.csv')

# Data Cleaning
data['Age'].fillna(data['Age'].median(), inplace=True)
data['Embarked'].fillna(data['Embarked'].mode()[0], inplace=True)
data.drop('Cabin', axis=1, inplace=True)

# Convert categorical features to numeric
data['Sex'] = data['Sex'].map({'male': 0, 'female': 1})
data = pd.get_dummies(data, columns=['Embarked'], drop_first=True)

# Write your code here for Box Plot for Age by Survived
```

```
data.boxplot(column='Age', by='Survived')
plt.title('Age by Survival')
plt.suptitle('')
plt.xlabel('Survived')
plt.ylabel('Age')
plt.show()
```

5.2.9. Box Plot for Fare by Pclass

```
import pandas as pd
import matplotlib.pyplot as plt

# Load the Titanic dataset
data = pd.read_csv('Titanic-Dataset.csv')

# Data Cleaning
data['Age'].fillna(data['Age'].median(),
inplace=True)
```

```
data['Embarked'].fillna(data['Embarked'].mode()[0], inplace=True)
data.drop('Cabin', axis=1, inplace=True)

# Convert categorical features to numeric
data['Sex'] = data['Sex'].map({'male': 0, 'female': 1})
data = pd.get_dummies(data, columns=['Embarked'], drop_first=True)

# Write your code here for Box Plot for Fare by Pclass
data.boxplot(column='Fare', by='Pclass')
plt.title('Fare by Pclass')
plt.suptitle('')
plt.xlabel('Pclass')
plt.ylabel('Fare')
```

```
plt.show()
```

5.2.10. Scatter Plot for Age vs. Fare

```
import pandas as pd
import matplotlib.pyplot as plt

# Load the Titanic dataset
data = pd.read_csv('Titanic-Dataset.csv')

# Data Cleaning
data['Age'].fillna(data['Age'].median(),
inplace=True)
data['Embarked'].fillna(data['Embarked'].m
ode()[0], inplace=True)
data.drop('Cabin', axis=1, inplace=True)

# Convert categorical features to numeric
```

```
data['Sex'] = data['Sex'].map({'male': 0, 'female': 1})
data = pd.get_dummies(data, columns=['Embarked'], drop_first=True)


# Write your code here for Box Plot for Fare by Pclass
plt.scatter(data['Age'], data['Fare'])
plt.title('Age vs. Fare')
plt.xlabel('Age')
plt.ylabel('Fare')
plt.show()
```

5.2.11. Scatter Plot for Age vs. Fare by Survived

```
import pandas as pd
import matplotlib.pyplot as plt
```

```
# Load the Titanic dataset
data = pd.read_csv('Titanic-Dataset.csv')

# Data Cleaning
data['Age'].fillna(data['Age'].median(), inplace=True)
data['Embarked'].fillna(data['Embarked'].mode()[0], inplace=True)
data.drop('Cabin', axis=1, inplace=True)

# Convert categorical features to numeric
data['Sex'] = data['Sex'].map({'male': 0, 'female': 1})
data = pd.get_dummies(data, columns=['Embarked'], drop_first=True)
```

```
# Write your code here for Scatter Plot for Age vs. Fare by Survived
colors = data['Survived'].map({0: 'red', 1: 'blue'})
plt.scatter(data['Age'], data['Fare'], c=colors)
plt.title('Age vs. Fare by Survival')
plt.xlabel('Age')
plt.ylabel('Fare')
plt.show()
```